

코드 개발 보안 정책

개발 코드 보안 검사 정책

AI 기반 소스코드 보안 검사 시스템용생성일: 2025-10-30 | 버전: 2.0

문서명:	개발 코드 보안 검사 정책 가이드라인 (Code Security Policy)
적용 대상:	소스코드, 스크립트, 설정파일, requirements.txt, docker-compose.yml 등
보안 등급:	Internal Use Only (사내 배포 전용)
목적:	하드코딩된 시크릿 차단, 보안 취약점 사전 방지, 안전한 코드 작성 가이드

1. 인증정보 하드코딩 검사

CRITICAL

1.1 API 키 및 액세스 토큰

코드에 API 키, 액세스 토큰, 시크릿 키 등이 직접 포함된 경우 **즉시 차단**합니다.

검사 패턴:

- OpenAI API Key: sk-[a-zA-Z0-9]{20,}
- AWS Access Key: AKIA[0-9A-Z]{16}
- AWS Secret Key: [a-zA-Z0-9/+=]{40}
- GitHub Token: ghp_[a-zA-Z0-9]{36}
- GitHub OAuth: gho_[a-zA-Z0-9]{36}
- Google API Key: AIza[0-9A-Za-z_-]{35}
- Stripe Key: sk_live_[a-zA-Z0-9]{24,}
- Slack Token: xox[baprs]-[a-zA-Z0-9-]{10,}
- JWT Token: eyJ[a-zA-Z0-9_-]*\.eyJ[a-zA-Z0-9_-]*
- Bearer Token: Bearer [a-zA-Z0-9_-]{20,}
- 변수명: api_key, access_token, secret_key, api_secret 등에 문자열 할당

❌ 위반 예시:

```
OPENAI_API_KEY = "sk-proj-abc123xyz789defghijk"
```

```
aws_access = "AKIAIOSFODNN7EXAMPLE"
```

```
github_token = "ghp_1234567890abcdefghijklmnopqrstuvwxyz"
```

```
stripe_key = "sk_live_51H4abc123xyz"
```

```
headers = {"Authorization": "Bearer abc123xyz"}
```

 올바른 방법:

```
api_key = os.getenv("OPENAI_API_KEY")
```

```
token = load_from_vault("github_token")
```

```
secret = get_secret("aws_secret_key")
```

CRITICAL

1.2 데이터베이스 인증정보

데이터베이스 연결 문자열, 비밀번호가 코드에 포함된 경우 차단합니다.

검사 패턴: - 연결 문자열: password=, pwd=, passwd= 포함 - MongoDB URI: mongodb://user:password@ - PostgreSQL: postgresql://user:password@ - MySQL: mysql://user:password@ - Redis: redis://:password@ - 변수명: db_password, database_password, mysql_pwd 등

 위반 예시:

```
DB_PASSWORD = "MySecretPass123!"
```

```
conn_str = "Server=db.com;Database=prod;Password=secret123"
```

```
MONGO_URI = "mongodb://admin:Pass1234@cluster0.mongodb.net"
```

```
REDIS_URL = "redis://:mypassword@localhost:6379"
```

✅ 올바른 방법:

```
db_password = os.getenv("DATABASE_PASSWORD")
```

```
conn_str = f"Server=x;Password={get_secret('db_pwd')}"
```

CRITICAL

1.3 암호화 키 및 시크릿

암호화 키, JWT 시크릿, OAuth 시크릿이 코드에 포함된 경우 차단합니다.

검사 패턴: - JWT_SECRET, SECRET_KEY 변수에 문자열 할당 - ENCRYPTION_KEY, CRYPTO_KEY 하드코딩 - OAuth CLIENT_SECRET 하드코딩 - 32자 이상의 랜덤 문자열 (키로 추정)

❌ 위반 예시:

```
JWT_SECRET = "my-super-secret-jwt-key-12345"
```

```
SECRET_KEY = "django-insecure-abc123xyz789"
```

```
ENCRYPTION_KEY = "a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6"
```

```
CLIENT_SECRET = "oauth_secret_abc123xyz"
```

CRITICAL

1.4 Private Key 및 인증서

SSH 키, SSL 인증서, PGP 키 등이 코드에 포함된 경우 차단합니다.

검사 패턴: - -----BEGIN PRIVATE KEY----- - -----BEGIN RSA PRIVATE KEY----- - -----BEGIN OPENSSH PRIVATE KEY----- - -----BEGIN PGP PRIVATE KEY----- - ssh-rsa AAAA..., ssh-ed25519 AAAA...

❌ 위반 예시:

```
private_key = ""-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEA...
-----END RSA PRIVATE KEY-----""
```

2. 보안 취약점 검사

HIGH

2.1 SQL 인젝션 취약점

사용자 입력을 SQL 쿼리에 직접 결합하는 경우 차단합니다.

검사 패턴: - f-string으로 SQL 조합: f"SELECT ... WHERE ... {variable}" - 문자열 연결: "SELECT * FROM" + user_input - .format()으로 SQL 조합 - 파라미터화되지 않은 쿼리 실행 - SQL 키워드 + 변수 조합 패턴

❌ 위반 예시:

```
query = f"SELECT * FROM users WHERE username = '{username}'"
```

```
sql = "DELETE FROM accounts WHERE id = " + user_id
```

```
cursor.execute("SELECT * FROM x WHERE name='" + name + "'")
```

```
query = "UPDATE users SET email='{ }' WHERE id={ }".format(email, id)
```

✅ 올바른 방법:

```
cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
```

```
query = "SELECT * FROM x WHERE id = %s"; cursor.execute(query, (user_id,))
```

```
User.objects.filter(username=username) # ORM 사용
```

HIGH

2.2 커맨드 인젝션

사용자 입력을 시스템 명령어에 직접 사용하는 경우 차단합니다.

검사 패턴: - `os.system()` 사용 - `subprocess.call()` with `shell=True` - `eval()`, `exec()`
사용 - f-string으로 명령어 조합

❌ 위반 예시:

```
os.system(f"rm -rf {user_path}")
```

```
subprocess.call(f"ping {hostname}", shell=True)
```

```
eval(user_input)
```

```
exec(f"result = {user_code}")
```

✅ 올바른 방법:

```
subprocess.run(["ping", hostname], shell=False)
```

```
# eval/exec 대신 ast.literal_eval 또는 안전한 파서 사용
```

HIGH

2.3 XSS (Cross-Site Scripting)

사용자 입력을 HTML에 이스케이프 없이 삽입하는 경우 차단합니다.

검사 패턴: - HTML 템플릿에 변수 직접 삽입: `f"<div>{user_input}</div>"` - `innerHTML`, `outerHTML` 사용 - `dangerouslySetInnerHTML` 사용 (React) - `render_template_string` with user input

❌ 위반 예시:

```
return f"<p>Welcome {user_name}!</p>"
```

```
html = f"<div class='comment'>{comment}</div>"
```

```
element.innerHTML = userInput;
```

```
<div dangerouslySetInnerHTML={{'__html': userData}} />
```

✅ 올바른 방법:

```
from markupsafe import escape  
return f"<p>{escape(user_name)}</p>"
```

```
element.textContent = userInput; # innerHTML 대신
```

HIGH

2.4 경로 조작 (Path Traversal)

사용자 입력을 파일 경로에 직접 사용하는 경우 차단합니다.

검사 패턴: - open(), with open() 사용 시 사용자 입력 포함 - os.path.join() with user input - 파일 경로에 ../ 검증 없음

❌ 위반 예시:

```
filename = request.GET.get('file')  
with open(f"/uploads/{filename}") as f:
```

```
path = os.path.join("/data", user_input)  
os.remove(path)
```

✅ 올바른 방법:

```
filename = secure_filename(request.GET.get('file'))  
if '..' not in filename:  
    with open(f"/uploads/{filename}") as f:
```

HIGH

2.5 비밀번호 평문 저장

비밀번호를 해시 처리 없이 저장하거나 약한 해시 사용 시 차단합니다.

검사 패턴: - password 변수를 DB에 직접 저장 - MD5, SHA1 해시 사용 (취약) - Base64 인코딩만 사용 - bcrypt, argon2, scrypt 등 강력한 해시 미사용

❌ 위반 예시:

```
db.insert({'username': user, 'password': password})
```

```
hashed = hashlib.md5(password.encode()).hexdigest()
```

```
encoded = base64.b64encode(password.encode())
```

```
hashed = hashlib.sha1(password.encode()).hexdigest()
```

✅ 올바른 방법:

```
import bcrypt
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
```

```
from werkzeug.security import generate_password_hash
hashed = generate_password_hash(password)
```

MEDIUM

2.6 민감정보 로그 출력

로그나 콘솔에 인증정보, 시크릿 등을 출력하는 경우 경고합니다.

검사 패턴: - print(), logger.info() 등에 민감 변수명 포함 - 키워드: password, token, secret, key, api_key, access_token - 환경변수 전체 출력: os.environ

❌ 위반 예시:

```
print(f"DB Password: {db_password}")
```

```
logger.info(f"API Key: {api_key}")
```

```
console.log("Token:", auth_token)
```

```
print(os.environ) # 전체 환경변수 노출
```

✅ 올바른 방법:

```
logger.info(f"DB connection established")
```

```
print(f"API Key: {api_key[:5]}****") # 일부만 표시
```

MEDIUM

2.7 입력값 검증 미흡

사용자 입력을 검증 없이 사용하는 경우 경고합니다.

검사 패턴: - request.GET, request.POST, request.args 직접 사용 - int(), float() 등에 try-except 없이 사용 - 타입 검증, 범위 검증 없음

❌ 위반 예시:

```
age = int(request.GET.get('age'))
```

```
price = float(request.POST.get('price'))
```

```
user_id = request.args.get('id') # 검증 없음
```

✅ 올바른 방법:

```
try:
    age = int(request.GET.get('age', 0))
    if not 0 <= age <= 150:
        raise ValueError
except ValueError:
    return error_response("Invalid age")
```

MEDIUM

2.8 HTTPS 미사용

민감 데이터 전송 시 HTTP 사용하거나 SSL 검증 비활성화 시 경고합니다.

검사 패턴: - http:// URL 사용 (production 환경) - verify=False (SSL 검증 비활성화) - requests.get(url, verify=False)

❌ 위반 예시:

```
response = requests.post("http://api.example.com/auth", json=data)
```

```
r = requests.get(url, verify=False) # SSL 검증 비활성화
```

```
urllib3.disable_warnings() # 경고 숨김
```

✅ 올바른 방법:

```
response = requests.post("https://api.example.com/auth", json=data)
```

MEDIUM

2.9 CSRF 보호 미적용

상태 변경 작업에 CSRF 토큰 검증이 없는 경우 경고합니다.

검사 패턴: - POST/PUT/DELETE 핸들러에 CSRF 검증 없음 - @csrf_exempt 데코레이터 사용 - Flask에서 WTForms 미사용

❌ 위반 예시:

```
@app.route('/delete-user', methods=['POST'])
def delete_user():
    user_id = request.form.get('id')
    User.delete(user_id) # CSRF 검증 없음
```

LOW

2.10 취약한 난수 생성

보안이 필요한 곳에 random 모듈 사용 시 알림합니다.

검사 패턴: - random.random(), random.randint() 사용 (보안 토큰 생성 시) - secrets 모듈 미사용

❌ 위반 예시:

```
token = str(random.randint(100000, 999999)) # 예측 가능
```

✅ 올바른 방법:

```
import secrets  
token = secrets.token_urlsafe(32)
```

3. 의존성 보안 검사

LOW

3.1 취약한 패키지 버전

알려진 보안 취약점이 있는 구버전 라이브러리 사용 시 알립니다.

검사 대상 (Python): - Django < 3.2 - Flask < 2.3 - requests < 2.31 - Pillow < 10.0
- urllib3 < 2.0 - cryptography < 41.0 - SQLAlchemy < 2.0 - Jinja2 < 3.1 **검사 대상**
(Node.js): - express < 4.18 - axios < 1.6 - lodash < 4.17.21

❌ 위반 예시 (requirements.txt):

```
Django==2.2.0 # CVE-2019-12308  
Flask==1.1.0 # 다수의 보안 패치 누락  
requests==2.18.0 # SSL 취약점  
Pillow==6.0.0 # 원격 코드 실행 취약점
```

✅ 올바른 방법:

```
Django>=4.2  
Flask>=2.3  
requests>=2.31  
Pillow>=10.0
```